

Cryptography Lab 2026 -- Report

Student number: 3035987788

Task 1 – Ps and Qs Attack

1. Workflow of the attack (1 mark)

The class's key generator reused the same broken random source, so at some point it handed out an identical prime to two different students while the other prime differed. This means their public moduli $N_1 = p \cdot q_1$ and $N_2 = p \cdot q_2$ share the factor p even though N_1 and N_2 look independent and random on their own. The attack proceeds in three steps:

- 1 **Detect the shared prime.** For every other classmate's modulus N_i in the roster, compute $g = \gcd(N, N_i)$ with the Euclidean algorithm. If $g = 1$ the two moduli are coprime and share nothing; if $g > 1$, g itself is a common prime factor (is_waldo returns True). Running this gcd against the whole class of 119 other students found exactly one classmate (student 3036292817 in my instance) whose modulus shares a prime with mine.
- 2 **Factor both moduli.** Once $g = \gcd(N_1, N_2)$ is found, it equals one of the two primes, say p . The other factor follows immediately by division: $q_1 = N_1 / p$. This single gcd computation, which is cheap even for 1024-bit numbers, fully factors a modulus that would otherwise take longer than the age of the universe to factor directly.
- 3 **Derive the private key.** With p and q known, compute $\lambda(N) = \text{lcm}(p-1, q-1)$ and invert the known public exponent e modulo $\lambda(N)$ using the extended Euclidean algorithm to get $d = e^{-1} \bmod \lambda(N)$. This d is then used to decrypt my ciphertext: $m = c^d \bmod N$, which reveals the flag CRYPTO{psqs_3035987788}.

2. Prevention in practice (1 mark)

The root cause is a weak or improperly reseeded random number generator that produces the same prime candidate more than once. It is prevented by: (a) always seeding key generation from a cryptographically secure source with enough entropy (e.g. the OS CSPRNG, /dev/urandom, never a fixed or restart-reset seed); (b) generating primes independently for every key and never caching or reusing prime material across keys or reboots; and (c) proactively screening newly generated moduli against a database of previously issued moduli with a batch-gcd check (this is exactly what large-scale audits such as the "Ron was wrong, Whit is right" and "Mining Your Ps and Qs" studies did to find this class of vulnerability in the wild).

Task 2 – Wiener's Attack

1. Continued fraction and convergents of 355/113 (2 marks)

Continued fraction expansion, by repeated division (Euclidean algorithm):

$$\begin{aligned} 355 &= 3 \cdot 113 + 16 \rightarrow a_0 = 3 \\ 113 &= 7 \cdot 16 + 1 \rightarrow a_1 = 7 \\ 16 &= 16 \cdot 1 + 0 \rightarrow a_2 = 16 \end{aligned}$$

so $355/113 = [3; 7, 16]$.

Convergents, using $h_{-1}=1, h_{-2}=0, k_{-1}=0, k_{-2}=1$ and $h_i = a_i h_{i-1} + h_{i-2}, k_i = a_i k_{i-1} + k_{i-2}$:

$$\begin{aligned} i=0, a_0=3: h_0 &= 3 \cdot 1 + 0 = 3, k_0 = 3 \cdot 0 + 1 = 1 \rightarrow 3/1 \\ i=1, a_1=7: h_1 &= 7 \cdot 3 + 1 = 22, k_1 = 7 \cdot 1 + 0 = 7 \rightarrow 22/7 \\ i=2, a_2=16: h_2 &= 16 \cdot 22 + 3 = 355, k_2 = 16 \cdot 7 + 1 = 113 \rightarrow 355/113 \end{aligned}$$

The convergents are $3/1, 22/7, 355/113$ — the last one recovers the original fraction exactly, as it must.

2. e/N approximates k/d (1 mark)

RSA's key equation is $ed \equiv 1 \pmod{\phi(N)}$, i.e. $ed = 1 + k\phi(N)$ for some integer k . Dividing both sides by $d \cdot \phi(N)$:

$$e/\phi(N) - k/d = 1/(d \cdot \phi(N))$$

so $e/\phi(N)$ is already extremely close to k/d . Now $\phi(N) = N - (p+q) + 1$, and since $p, q \approx \sqrt{N}$ for a balanced RSA modulus, $\phi(N)$ differs from N by only about $2\sqrt{N}$ — a tiny relative amount for a 1024-bit N . Substituting N for $\phi(N)$ therefore changes the approximation only slightly, so the *public* ratio e/N is also an excellent approximation of the *secret* fraction k/d :

$$e/N - k/d \approx e/\phi(N) - k/d = 1/(d \cdot \phi(N)) \ll 1$$

3. Legendre's condition holds when $d < N^{1/4}/3$ (2 marks)

Legendre's theorem says any rational k/d with $|e/N - k/d| < 1/(2d^2)$ must be a convergent of e/N .

Starting from $e/N - k/d = [1 - k(p+q-1)] / (Nd)$ (obtained by putting $ed = 1 + k\phi(N)$ and $\phi(N) = N - (p+q) + 1$ over the common denominator Nd), and using that $k < d$ (since $k = (ed-1)/\phi(N) < e/\phi(N) < d$ for $e < \phi(N)$) together with the standard bound $p+q < 3\sqrt{N}$ for two primes of similar size:

$$|e/N - k/d| < k(p+q) / (Nd) < d \cdot 3\sqrt{N} / (Nd) = 3/\sqrt{N}$$

We want this smaller than the Legendre bound $1/(2d^2)$:

$$3/\sqrt{N} < 1/(2d^2) \Leftrightarrow d^2 < \sqrt{N} / 6 \Leftrightarrow d < N^{1/4} / \sqrt{6} \approx 0.408 \cdot N^{1/4}$$

Since $1/3 \approx 0.333$ is smaller (stricter) than $1/\sqrt{6} \approx 0.408$, the condition $d < N^{1/4}/3$ stated in the task is a sufficient (slightly conservative) bound that guarantees $3/\sqrt{N} < 1/(2d^2)$, so $|e/N - k/d| < 1/(2d^2)$ holds and Legendre's theorem forces k/d to appear among the convergents of e/N — exactly what `wiener_attack()` searches. In my instance the recovered d was 144 bits long against a 1024-bit N , comfortably inside this bound, and it decrypted to `CRYPTO{w1ener_3035987788}`.

Task 3 – Håstad's Broadcast Attack

1. How CRT combines the ciphertexts, and why 3 receivers are enough for $e=3$ (2 marks)

Each receiver computes $C_i = m^3 \bmod N_i$ for the *same* underlying message m , using three pairwise coprime moduli. This is exactly a system of simultaneous congruences $x \equiv C_i \pmod{N_i}$ in the single unknown $x = m^3$. The Chinese Remainder Theorem guarantees this system has a unique solution modulo $M = N_0 N_1 N_2$, computed as

$$x = \sum_i C_i \cdot M_i \cdot (M_i^{-1} \bmod N_i) \bmod M, \quad M_i = M / N_i.$$

Because the flag m is a short string (much smaller than any single 1024-bit N_i), m^3 is smaller than $M = N_0 N_1 N_2$ (roughly 3072 bits), so the CRT solution x is not just congruent to m^3 modulo M — it *is* m^3 exactly, with no wraparound. Taking the ordinary integer cube root of x then recovers m with no need for any private key. Two ciphertexts are not enough because with only $N_0 N_1$ (about 2048 bits) there is no guarantee that this smaller product exceeds m^3 (m^3 can be up to roughly $3 \cdot 1024 = 3072$ bits for a message comparable in size to one modulus); the CRT result would then only pin down m^3 modulo $N_0 N_1$, leaving infinitely many integers $x, x + N_0 N_1, x + 2N_0 N_1, \dots$ consistent with the two congruences, none of which can be singled out as the true m^3 by taking a cube root. In general, for public exponent e you need e ciphertexts so that the product of e moduli exceeds m^e — hence the name broadcast attack.

2. Why fresh random padding defeats the attack (1 mark)

The whole attack relies on the three ciphertexts encoding the *same* integer m , so that $C_i = m^3 \bmod N_i$ are three views of one unknown and CRT can combine them into it. If a fresh random padding string is prepended (or otherwise mixed in) independently before each encryption, the three receivers actually encrypt three different integers m_0, m_1, m_2 ($\text{pad}_i \parallel \text{flag}$ as an integer). CRT combining C_0, C_1, C_2 under this assumption no longer corresponds to any single well-defined cube, so the combined value is not a perfect cube and no integer cube root recovers a meaningful plaintext. This is precisely why real-world RSA encryption always uses randomized padding schemes such as OAEP (or at least PKCS#1 v1.5).

Task 4 – Hash Attacks

1. Why length extension works, and what the attacker needs beforehand (2 marks)

MD5, like all Merkle–Damgård hashes, processes the padded message in fixed-size blocks, updating a small internal chaining state (a, b, c, d) one block at a time, and simply *outputs that final state* as the digest — there is no separate finalisation step that mixes in secret information. This means $\text{MAC} = \text{MD5}(\text{key} \parallel \text{msg})$ is exactly the internal chaining state that existed right after the last block of $\text{key} \parallel \text{msg} \parallel \text{padding}$ was compressed. An attacker who captures this MAC can therefore treat it as a valid starting state and resume the exact same compression function on more 64-byte blocks, computing $\text{MD5}(\text{key} \parallel \text{msg} \parallel \text{padding} \parallel \text{extra})$ for any suffix extra of their choice — all without ever learning the key itself. In my forged request the captured MAC on $\text{msg} = \text{'user=425969\&action=read'}$ ($\text{key_len} = 23$) was extended with $\text{extra} = \text{'\&action=admin'}$ to produce a forged message and a forged MAC that the server would accept as genuine. Concretely, the attacker needs, before the forgery: (a) at least one valid (message, MAC) pair produced with the target key, and (b) the *length* of the secret key (not its value) — needed only to compute where the Merkle–Damgård padding is inserted, since padding depends on the total length $\text{key_len} + \text{len}(\text{msg})$.

2. Why HMAC should be used instead of MD5(key||message) (1 mark)

HMAC nests two hash calls: $H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$. Its output is not simply "the internal state after hashing a longer prefix" the way MD5(key||msg) is: the outer hash is computed over the already-finished inner digest concatenated with a key-derived block, so there is no exposed intermediate chaining state that an attacker who only sees the final HMAC tag can resume computation from — continuing the outer hash would require knowing $K \oplus \text{opad}$, which requires the key. This structural difference is exactly what defeats the length-extension attack that plain MD5(key||message) is vulnerable to.

3. Birthday-bound estimates for 40-bit and 128-bit digests (2 marks)

By the birthday bound, a 50% chance of finding a collision among random n -bit digests needs about $1.18 \cdot 2^{n/2}$ samples (the classical birthday-paradox approximation $\sqrt{2 \cdot \ln 2} \cdot 2^{n/2} \approx 1.18 \cdot 2^{n/2}$):

$n = 40$ bits: $1.18 \cdot 2^{20} \approx 1.24 \times 10^6$ messages — about 1.2 million tries, found in well under a second in my find_collision() implementation (birthday-attack collision search using a dictionary keyed by the 5-byte truncated digest).

$n = 128$ bits: $1.18 \cdot 2^{64} \approx 2.18 \times 10^{19}$ messages — utterly infeasible with current computing power, which is why the full 128-bit MD5 digest (and larger digests such as SHA-256's 256 bits) give meaningful collision resistance while a 40-bit truncation does not.

All four self-tests (task1–task4, run via python3 test.py in each folder) pass with full marks, and each recovered flag contains my student number (3035987788), confirming the attacks above were carried out against my own per-student data rather than copied values.